

Project-Kickoff

Kursüberblick



1. Intro
2. Mobile UX & Input
3. Performance & Mobile Constraints
4. GPS & Location-Based Mechanics
5. Map Integration
6. AR Foundation
7. Game Loop & Systems
8. Polishing + Smartphone Features
9. Prüfung

Intro

- Projekt Setup
 - Unity
 - adb
- Android Build Pipeline in Unity:
 - Developer Mode auf Android,
 - Build & Deploy auf echte Geräte,
 - Logcat + Debugging
- Besonderheiten von Mobile Games vs. PC/Console:
 - IL2CPP vs. Mono, Gradle,
 - APK vs AAB,
 - weitere

Was braucht ihr?

- Laptop mit
 - Windows (empfohlen, Linux/Apple geht auch, ist aber Hardmode)
 - Unity3D 6.4 (empfohlen) -> **jetzt herunterladen!**
 - Modules: Android Build Support, *OpenJDK*, *Android SDK & NDK Tools*
- GitHub Account
 - <https://github.com/Tillomaticus/MobileGameProg26Assets>
- Android Smartphone / Leihgerät mit $\geq$ Android 7.1 (Nougat)
 - Android-Simulator geht aber auch
 - USB Kabel, passend zum Android Gerät und eurem Laptop
- adb (Android Debug Bridge) -> Android Tools (*später gemeinsam*)

(optional) Android Studio - für den Android Device Emulator (statt echtem Gerät)

Projekt Setup - Lästiger Kram

1. Unity 3D starten
 - a. neues Projekt -> Universal 2D
2. Github Projekt einrichten
3. adb herunterladen -> Command line tool für die Kommunikation mit dem angeschlossenen Android device
 - a. Google -> Android SDK platform tools -> Download
 - b. Bitte **nicht** im Ordner "Download" lassen! Stattdessen verschieben.
 - i. z.B. "C:/Benutzer/\$name/platform-tools"

Projekt Setup II - adb

1. ADB als Umgebungsvariable setzen:
 - a. dazu in Windows Suche: “*View Advanced System Settings*” -> Erweiterte Systemeinstellungen anzeigen -> Umgebungsvariablen
 - b. bei Systemvariablen (unteres Feld) -> Path anklicken -> bearbeiten
 - c. Neu -> den Pfad der adb.exe dort rein: z.B. “C:\Users\\$name\platform-tools”
2. Kommandokonsole öffnen. “adb” eingeben
 - a. adb sollte erkannt werden. Wenn nicht, dann schreien: “**Du Scheissgerät!**”
3. (bonus) Android - Entwicklermodus einschalten
 - a. Telefoninfo -> Softwareinfo -> 7x auf Buildnummer tippen
 - b. Entwickleroptionen tauchen auf, dort prüfen ob USB-Debugging eingeschaltet ist.
4. Android Gerät am PC anschließen
 - a. USB Debugging zulassen (am besten Haken setzen bei “immer zulassen”)
5. Kommandokonsole öffnen. “adb devices” eingeben
 - a. Device wird angezeigt. Wenn nicht, dann schreien: “**Du Scheisshandy!**”

Projekt Setup III - Lästiger Kram in Unity

1. Preferences überprüfen:

- a. Edit -> Preferences -> External Tools:
 - i. Stimmt der Scripting Editor?
 - ii. Sind bei Android die Packages angehakt und zeigen auf die richtige Editor Version?

2. Build Profile auf Android stellen

- a. File -> Build Profiles -> Add Build Profile -> Android

3. Project Settings setzen

- a. Edit -> Project Settings -> Player:
 - i. Company Name, Product Name. -> wichtig, daraus generiert sich euer Package Name!
 - ii. Other Settings: Identification: Override Default Package Name -> aus.
- b. TextMesh Pro -> Import TMP Essentials
- c. Graphics -> Default Render Pipeline -> Universal RP Asset einfügen

First Steps - Cooler Kram in Unity

Erste Aufgabe:

- Sprite in der Szene
- im Sekundentakt die Farbe wechseln (Frame unabhängig!)
- Text auf dem Sprite (TextMeshPro) : mit Name der **vorherigen** Farbe
- TextDebug.Log bei jedem Farbwechsel mit:
 - **Zeit seit Spielstart,**
 - **Name der alten Farbe,**
 - **Name der neuen Farbe**

Zeit: 45min

Bonus: Editierbarkeit der Farben und Namen im Inspector
(erst wenn die Basis funktioniert!)

Tipps

Farbe / Textverknüpfung:

```
Color[] _colors = { Color.red, Color.green, Color.blue, Color.cyan, Color.magenta, Color.yellow };  
string[] _colorNames = { "Red", "Green", "Blue", "Cyan", "Magenta", "Yellow" };
```

Time

```
Time.deltaTime; //Zeit seit dem letzten Frame, in Sekunden  
  
Time.realtimeSinceStartup //Zeit in Sekunden seit Gamestart
```

Für Inspector

```
[System.Serializable]  
public class NamedColor  
{  
    public Color color;  
    public string name;  
}
```

Lösungsbeispiel

```
float _timer = 0f;  
string _oldColorName = "white";
```

```
Color[] _colors = {  
    Color.red, Color.green, Color.blue,  
    Color.cyan, Color.magenta,  
    Color.yellow };
```

```
string[] _colorNames = { "Red",  
    "Green", "Blue", "Cyan", "Magenta",  
    "Yellow" };
```

```
[SerializeField]  
SpriteRenderer _spriteRenderer;
```

```
[SerializeField]  
TextMeshPro _textMeshPro;
```

```
void Update()  
{  
    //gibt die Zeit seit dem letzten Frame  
    aus  
    _timer += Time.deltaTime;  
    if (_timer >= 1f) {  
        _timer -= 1f;  
        TriggerColorSwitch();  
    }  
}
```

```
void TriggerColorSwitch () {  
    //Random Color generieren  
    int randomColor = Random.Range(0, _colors.Length);  
  
    //Farbe auf Sprite malen  
    _spriteRenderer.color = _colors[randomColor];  
  
    //neuen Farbnamen speichern  
    string newColorName = _colorNames[randomColor];  
  
    //Text im Sprite mit Namen der alten Farbe setzen  
    _textMeshPro.text = _oldColorName;  
  
    //Debug Log schreiben  
    Debug.Log("Time: " + Time.realtimeSinceStartup  
        + " old Color: " + _oldColorName + " new Color: "  
        + newColorName);  
  
    //Name der neuen Farbe für nächstes mal speichern  
    _oldColorName = newColorName;  
  
}
```

Kurze Pause?

Vom Editor zum Android Device - Simulation

- Device Simulator (pre-installed)
 - Package Manager -> Unity Registry -> Device Simulator Devices

1. Simulator-Ansicht:

- a. Game-View -> Dropdown-Menü "Game" -> Simulator

2. Euer Gerät suchen und auswählen:

- *"Ahhhh! Mein Gerät gibt es nicht!!"* -> Japp. Meins auch nicht.
- Unity baut gerade ein neues Tool und hat den Support eingestellt... Und jetzt?

Dann schreiben wir unser eigenes Gerät!

Eigenes Simulator Device

1. Ich hab da mal was vorbereitet -> Package runterladen (nur .unitypackage)
https://github.com/Tillomaticus/MobileGameProg26/releases/tag/Pixel6a_Simulator_Device
2. Assets -> Import Package -> Custom Package
3. Im Projektordner: Editor->Devices:
 - a. Google Pixel 6a duplizieren & in euer Device umbenennen
 - b. Mit Code Editor öffnen
 - c. Das ist eine JSON Datei die die Daten für das Gerät enthält -> hier Pixel6a
 - d. Jetzt KI nutzen um die Daten des Pixel6a mit eurem Gerät zu ersetzen.
 - e. Checken ob es grob passt. Aber Vorsicht:
Zeile 89 "overlayPath": "DeviceOverlays/Google Pixel 5_Overlay.png",
Dieser Pfad zeigt auf das Overlay. Hier sollte ein gültiger Pfad stehen.
 - f. (Optional) Overlay aus dem Unity Package kopieren oder einfach das Pixel 5 Overlay lassen.
 - g. Im Simulator nach eurem Gerät suchen und es aktivieren
 - h. Play drücken und zuschauen!

Building the Build - Warum jetzt schon?

Simulation läuft. Reicht doch?

Nein. Simulation bildet nie alles ab. Fehler z.B. bei:

- Build-Prozess
- Geräteeigenschaften
- Render-Probleme
- Architekturprobleme

daher: **“Fail fast, fail often”** - möglichst früh Builden und App testen. Im Zweifel gleich als erstes.

-> Ich spreche aus Erfahrung: Probleme mit Switch von Rift-S (PC) auf Quest

Vom Editor zum Android Device - Building the Build

Also, ab aufs richtige Gerät!

1. Im Build Profile

- a. Wir wollen Logs sehen, deswegen: Development Build anmachen!
- b. Haken bei Autoconnect Profiler setzen
- c. Build -> apkname setzen und speichern.
- d. hoffen, das der Build durchgeht! Wenn nicht, schreien: “Du Scheissbuild!”

Build ging durch? Gut.

2. Im Ordner mit dem Build -> Shift + Rechtsklick -> Powershell hier öffnen / in Terminal öffnen
3. `adb install -r <buildname>.apk`, auf Success warten.
4. App auf dem Gerät suchen und starten

Android Device - Logging mit Logcat

Wir wollen die Logs sehen!

1. cmd: **adb logcat**
 - a. Speedreading! Okay, doch besser anders. Ctrl+C zum stoppen.
2. cmd: **adb logcat > log.txt**
 - a. log.txt im Build Ordner öffnen
 - b. Immer noch ziemlich unübersichtlich... -> Ctrl+C
3. Jetzt mit Filter: **cmd: adb logcat -s Unity > log.txt**
4. Und jetzt nur noch Debug Logs: **cmd: adb logcat -s Unity:D > log.txt**
 - a. Das ist doch schon lesbarer.
 - b. Unseren Unity Log finden. Stimmen die Werte?

Android Device - Logging mit Unity

Okay, aber Konsole ist schon echt Oldschool... Wir haben doch Unity!

1. Unity wieder öffnen -> Build Profiles:
 - a. Bei Run Device unser Gerät auswählen
 - b. Build & Run klicken
 - c. selber Build-Name -> Build sollte viel schneller gehen als beim ersten mal.
2. Der Profiler öffnet sich automatisch -> fürs erste schließen, schauen wir uns ein anderes mal an
3. Die Unity Konsole spuckt unsere Logs aus! Hurra!

Kurze Pause?

Besonderheiten von Mobile vs. PC

- IL2CPP vs. Mono,
- APK vs AAB,
- Input, UI-Besonderheiten, Battery,
- Thermal Throttling,

IL2CPP vs Mono

Zwei Unterschiedliche Scripting-Backends (Runtime-Technologien)

Mono

- “just-in-time” (JIT)-compiler.
 - Führt den C# code direkt während der runtime aus

IL2CPP - (Intermediate Language to C++)

- vorkompiliert, während building
 - in .NET Intermediate Language
 - dann weiter kompiliert in C++,
 - dann weiter kompiliert in native binaries (z.B. ARM64)

Mono

Vorteile:

- schnellere Buildzeiten, da nichts umkompiliert werden muss.
- kleinere Buildgrößen
- leichtes Debugging
 - der Debugger kann direkt angehängt werden
 - Variablen können inspiziert werden
 - "hot-reload" möglich. -> wie im Editor

Nachteile:

- schlechtere Performance,
 - da der erst Code während der Runtime interpretiert wird
 - begrenzte Optimierungen für Methodenzugriffe und Pfade
 - relativ einfacher Garbage Collector, kann zu häufigen Pausen führen -> FPS drop
 - mehr Overhead um dynamische Features (z.B. Reflection und Delegates) zu ermöglichen

IL2CPP

Vorteile:

- bessere Performance,
 - vorkompiliert, dadurch kann der Compiler den gesamten Code analysieren und
 - Code-Pfade optimieren
 - Speicher optimieren
 - schnellere Ausführung, Code kann direkt auf der CPU laufen
- schwieriger zu reverse-engineeren, weil Maschinencode

Nachteile:

- längere Buildzeiten
- größere Buildfiles
- schwereres Debugging

Was nehmen wir?

Mono

- besser für Entwickler, schlechter für Spieler

IL2CPP

- schlechter für Entwickler, besser für Spieler

Und wo stellen wir das ein?

Project Settings -> Player Settings -> Other Settings -> Configuration

- Scripting Backend,
- Standard ist IL2CPP
- schauen wir uns später nochmal an

Kompiliert wird, aber in was?

Auf dem PC gibt es .exe Files, aber was gibt es bei mobile?

Android

- APK
- AAB

iOS

- Xcode

APK

APK - Android Package Kit

- eine gezippte Datei mit bestimmter Struktur und bestimmten Dateien
 - `build.apk` öffnen und anschauen (z.B. mit 7zip)

Enthält:

- `AndroidManifest.xml` -> Metadaten, Berechtigungen
- `classes.dex` -> kompilierter Code
- `res` -> kompilierte Ressourcen
- `lib` -> native Bibliotheken
- `Assets` -> != Unity Asset Ordner; aber enthält viel Unity Assets
- `META-INF` -> Signaturinformationen

APK

Statt einer .exe für den PC baut man für Android die .apk

- “single installable file”, alles drinn!
- **Vorteil:** kann direkt auf dem Gerät installiert werden,
 - via Sideloadung (adb)
 - via download & install
- **Nachteil:** Größere Datei, da Daten für alle (unterstützten) Gerättypen enthalten sein müssen
 - verschiedene CPU Architekturen
 - verschiedene Pixeldichten und Auflösungen, Texturkompressionsarten
 - verschiedene Sprachen

AAB

AAB - Android App Bundle

- seit August 2021 Pflicht für Apps im Play Store
- auch gezippte Datei, mit leicht anderer Struktur -> `build.aab` anschauen

AAB enthält keine Geräte-Dateien

- App Store weiß welches Gerät welche “Zusatzdateien” braucht
 - dadurch 20-50% kleine Downloads/Filesize
- Kein direktes Installieren mehr möglich, da die gerätespezifischen Dateien fehlen

Wann was?

APK

- für lokale Builds und Tests
- Für Apps die direkt verteilt werden -> z.B. über Device Management Software

AAB

- für alle Builds die im App Store hochgeladen werden sollen.

APK vs AAB Workflow in Unity

Für uns:

1. apk builden, lokal testen.
2. fertig

Für Releases im Appstore:

1. apk building, lokal testen.
2. im Android Build Profile Haken bei “**Build App Bundle (Google Play)**” setzen,
3. Build drücken und im App Store hochladen
4. fertig

Und Iphones?

Nur mit Mac möglich!

1. auf iOS Build Profile umstellen und builden
2. Projekt in Xcode öffnen
3. Build + Sign (benötigt Apple Developer Account)
4. Lokal Installieren / App Store

Was ist noch wichtig für einen Android Build?

Player Settings:

- Icons
- Resolution and Presentation
- Other
- Publishing

Icons

- Icon für eure App
- verschiedene Größen / Auflösungen
- verschiedene API Varianten
- Default ist das Unity Icon

Resolution and Presentation

für Android eher begrenzt

das wichtigste:

- Default Orientation
 - In welcher Orientation die App gestartet werden soll
- Allowed Orientations,
 - welche erlaubt sind
- Render Over Native UI
 - rendert das Game hinter / vor der normalen Android UI

Other

Identification

- Package Name -> eindeutige Identifizierung eurer App im Store
- Version -> “menschlesbare” Version der App,
 - Major, Minor, Patch
- Bundle Version Code -> Android only, ID für die Identifizierung des Builds,
 - man kann immer nur auf eine höhere Version updaten
- Minimum API Level
 - niedrigste Android Version, auf der die App installiert werden kann
 - definiert das “minimum Feature-Set”, dass die App benötigt
- Target API Level
 - die Version für die wir die App optimieren
 - neuere Android-Versionen wenden Kompatibilitäts-Funktionen an

Configuration

- Scripting Backend -> Hallo IL2CPP

API Level - Geräteschaft

Absolut

Platform Version	API	Distribution
Android 5 (Lollipop)	21	0.1%
Android 5.1 (Lollipop)	22	0.3%
Android 6 (Marshmallow)	23	0.4%
Android 7 (Nougat)	24	0.4%
Android 7.1 (Nougat)	25	0.4%
Android 8 (Oreo)	26	0.8%
Android 8.1 (Oreo)	27	2.3%
Android 9 (Pie)	28	4.5%
Android 10 (Q)	29	7.8%
Android 11 (R)	30	13.7%
Android 12 (S)	31	11.4%
Android 13 (T)	33	13.9%
Android 14 (U)	34	17.2%
Android 15 (V)	35	19.3%
Android 16 (B)	36	7.4%

Kumulativ

Platform Version	API	Distribution
Android 5 (Lollipop)	21	99.9%
Android 5.1 (Lollipop)	22	99.8%
Android 6 (Marshmallow)	23	99.5%
Android 7 (Nougat)	24	99.1%
Android 7.1 (Nougat)	25	98.7%
Android 8 (Oreo)	26	98.3%
Android 8.1 (Oreo)	27	97.5%
Android 9 (Pie)	28	95.2%
Android 10 (Q)	29	90.7%
Android 11 (R)	30	82.9%
Android 12 (S)	31	69.2%
Android 13 (T)	33	57.8%
Android 14 (U)	34	43.9%
Android 15 (V)	35	26.7%
Android 16 (B)	36	7.4%

<https://composables.com/android-distribution-chart>

Publishing

Publishing Settings

- Keys zum signieren der Apps
 - Anlegen von Keys
 - einer für die Organisation
 - einer für die App
 - Signieren der Apps beim Build
 - Wenn der Key verloren geht, kann die App nicht mehr geupdatet werden!
- es dürfen nur signierte Apps im Store hochgeladen werden
- signierte Apps können nur durch vom selben Autor signierte Apps geupdatet werden
- Für unsere lokalen Builds nicht nötig!

Split Application Binary

- erzeugt zusätzlich zur .apk eine/mehrere .obb
- Play Store hat 100MB Limit für die apk
- OBB können nachträglich geladen werden
- OBB können separat gepatcht werden ohne das gesamte Spiel neu installieren zu müssen

Weitere Besonderheiten

Input

- Keine Peripherie, limitierte Eingabemöglichkeiten

UI

- Landscape & Portrait, Safe Areas

Battery

- Strom auf Mobile ist nicht "for free"
- Mehr Leistung -> höherer Stromverbrauch -> kürzerer Spielspaß

Thermal Throttling

- Kompakte Geräte, fast ausschließlich passiv Kühlung
- Laptops werden auch heiß, muss aber keiner in der Hand halten
- Smartphones drosseln daher die Leistung, wenn sie heiß werden

➡ Nächste Stunde

➡ Übernächste
Stunde